Práctica de Introducción a la Concurrencia: "Carrera de Animales"

Objetivo

Simular una carrera entre varios animales usando hilos, donde los alumnos puedan:

- Crear y lanzar hilos con Runnable y Thread.
- Usar sleep() para simular tiempos variables.
- Sincronizar el acceso a una "meta" compartida con synchronized.
- Gestionar concurrencia con un Semaphore.
- Aplicar un patrón **Master–Worker** (el juez controla y los animales compiten).
- Observar el orden de finalización con join() o estructuras compartidas.

Enunciado

Crea una aplicación en Java que simule una **carrera entre 3 animales** (por ejemplo: tortuga, liebre, zorro). Cada animal es un hilo que avanza por tramos y anuncia su progreso. El primero que llegue a la meta debe ser anunciado como **ganador**, evitando empates mediante sincronización.

Estructura del proyecto

Copiar código

CarreraAnimales/

├— Animal.java

├— JuezCarrera.java

└— CarreraMain.java

Detalles técnicos que deben aplicar

- Usar una clase Animal que implemente Runnable.
- En el método run() que cada animal recorra 10 tramos con sleep() aleatorio (simula velocidad).
- Cuando lleguen al tramo 10, llamarán a un método llegarMeta() de la clase JuezCarrera.
- Este método debe estar **sincronizado** o controlado con **Semaphore(1)** para evitar que dos animales sean declarados ganadores.
- Mostrar en consola:
    - Progreso tramo a tramo.
    - Qué animal ganó.
    - Qué animal terminó segundo/tercero.

Ejemplo de ejecución esperada (salida consola)

[Tortuga] avanzando tramo 1...

[Liebre] avanzando tramo 1...

[Zorro] avanzando tramo 1...

[Liebre] avanzando tramo 2...

...

[Liebre] ha llegado a la meta 🏁

>> ¡Ganador: Liebre! 🥇

[Tortuga] ha llegado a la meta 🏁

[Zorro] ha llegado a la meta 🏁

>> Posiciones:

1. Liebre

2. Tortuga

3. Zorro

Criterios de evaluación

Requisito técnico	Peso
Uso correcto de Runnable y creación de hilos	20%
Simulación de tramos con sleep()	10%
Acceso sincronizado a la meta (synchronized o Semaphore)	25%
Seguimiento ordenado (con join() o estructura lista)	20%
Organización del código y claridad	15%
Salida visual clara en consola	10%

Extensiones (optativo)

- Añadir un BlockingQueue donde los animales reporten su estado.
- Registrar tiempos de llegada (System.currentTimeMillis()).
- Hacer que el "Juez" sea un hilo que observe la carrera.

Clases que debe haber y que se deben entregar:

1. Clase JuezCarrera.java

---

2. Clase Animal.java

---

3. Clase principal CarreraMain.java
